
Introducción al Aprendizaje Automático

1er cuatrimestre 2025



Universidad
Nacional
de San Martín

k vecinos más cercanos

KNN - K Nearest Neighbors

Recordamos que un buen clasificador es aquel que muestra una media de tasa de error baja

$$\text{Ave}(I(y_o \neq \hat{y}_o))$$

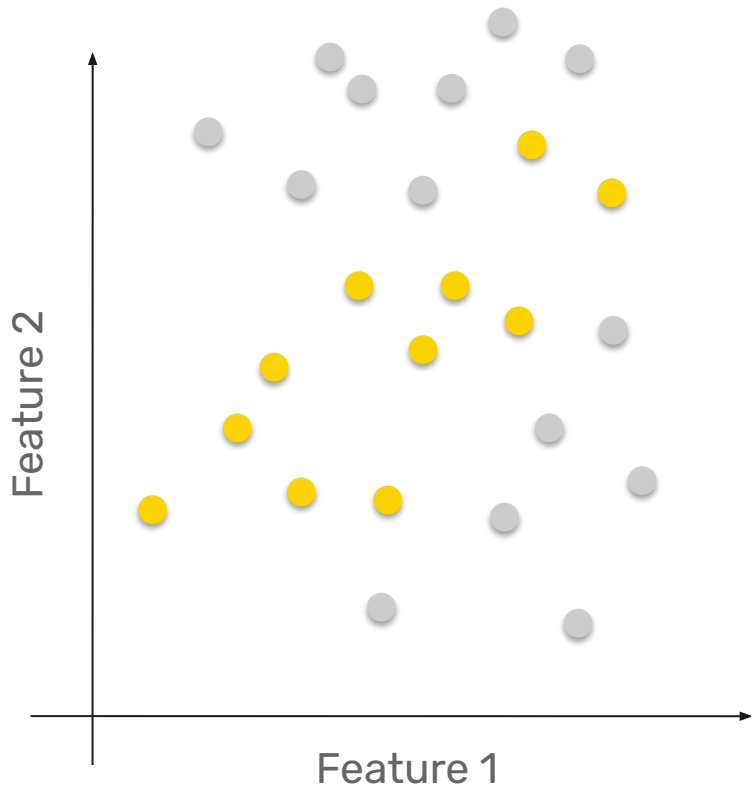
Y se minimizará si asigna cada observación a la clase más probable, dados los valores de las X

$$\text{Pr}(y=j/X=x_o)$$

IDEA: Dada una nueva instancia de la cual no conocemos la etiqueta objetivo, vamos a asumir que su etiqueta será igual a la de las instancias “vecinas” en el training set. *Clasifica una observación con la clase de probabilidad estimada más alta.*

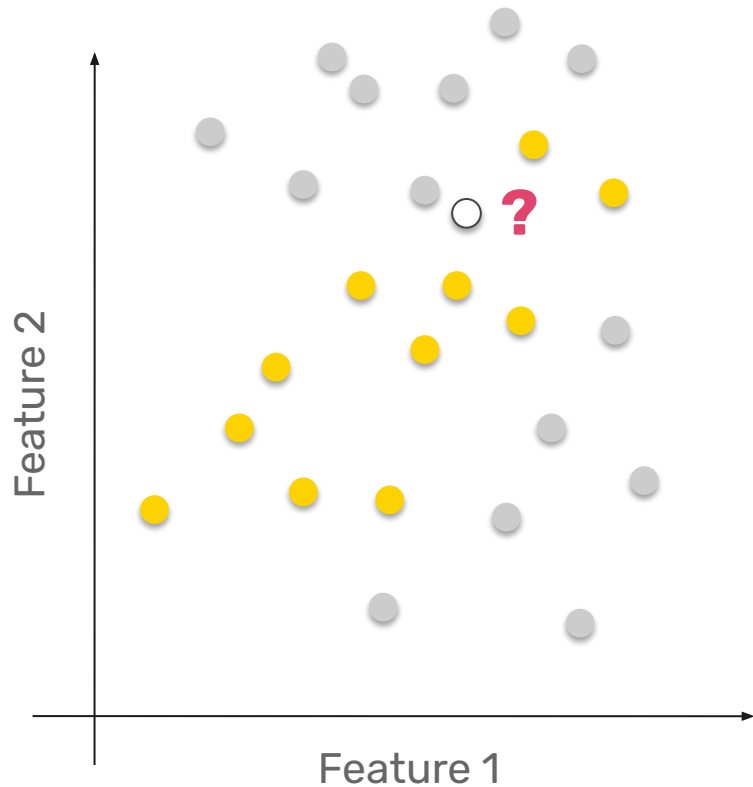
KNN Clasificación - Ejemplo

Supongamos que tenemos un Dataset con 2 Features, en el cual cada instancia puede pertenecer a una de dos clases: "Gris" o "Amarillo".



KNN Clasificación - Ejemplo

Dada una nueva instancia, de la cual no conocemos su clase, vamos a recurrir a sus vecinos para clasificarla.

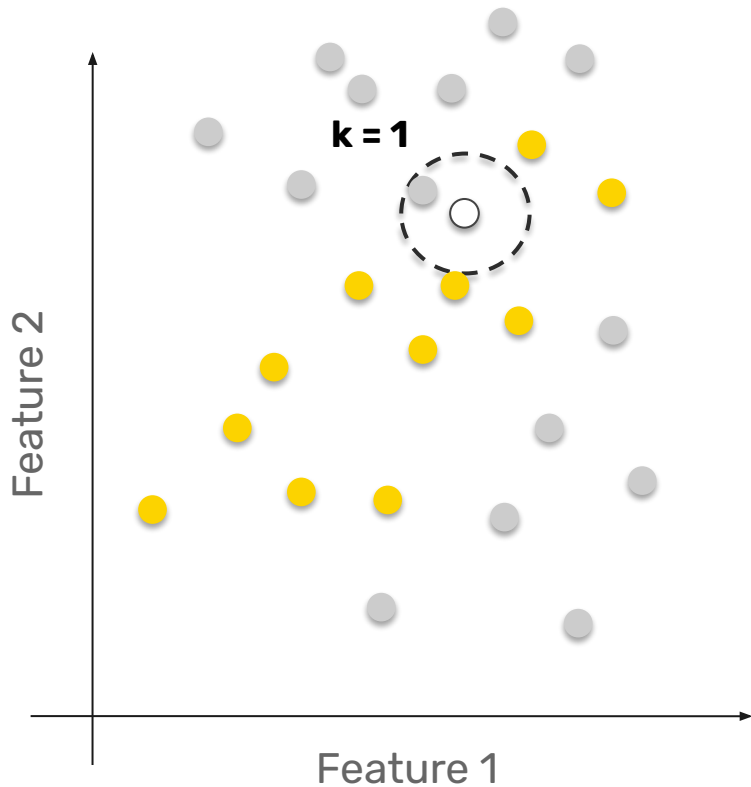
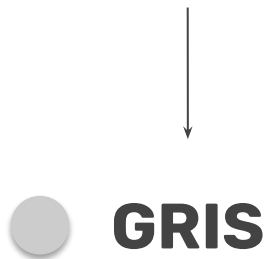


KNN Clasificación- Ejemplo

¿Por K en nombre del algoritmo?

K es el número de vecinos que miramos para saber la clase de nuestra nueva instancia.

Si tomamos **K=1**, solo miraremos al vecino más cercano.



KNN Clasificación- Ejemplo

Por ejemplo, con $k = 3$ tenemos dos vecinos Amarillos y uno Gris.

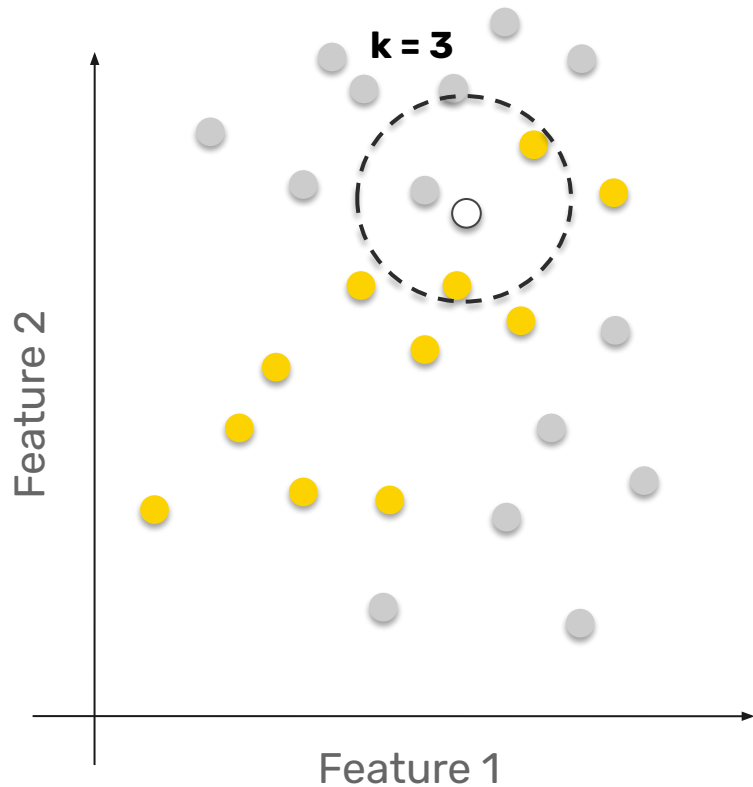


● **AMARILLO**

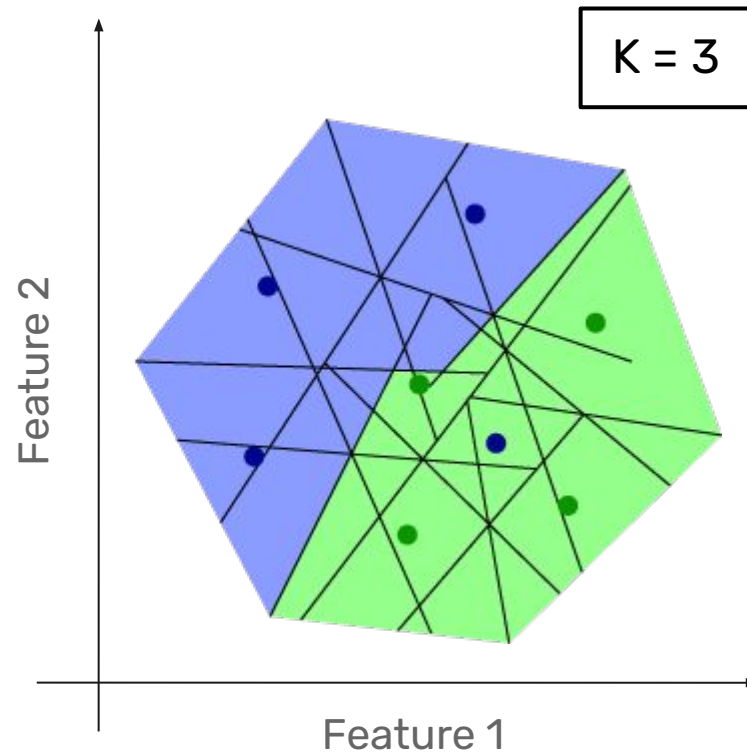
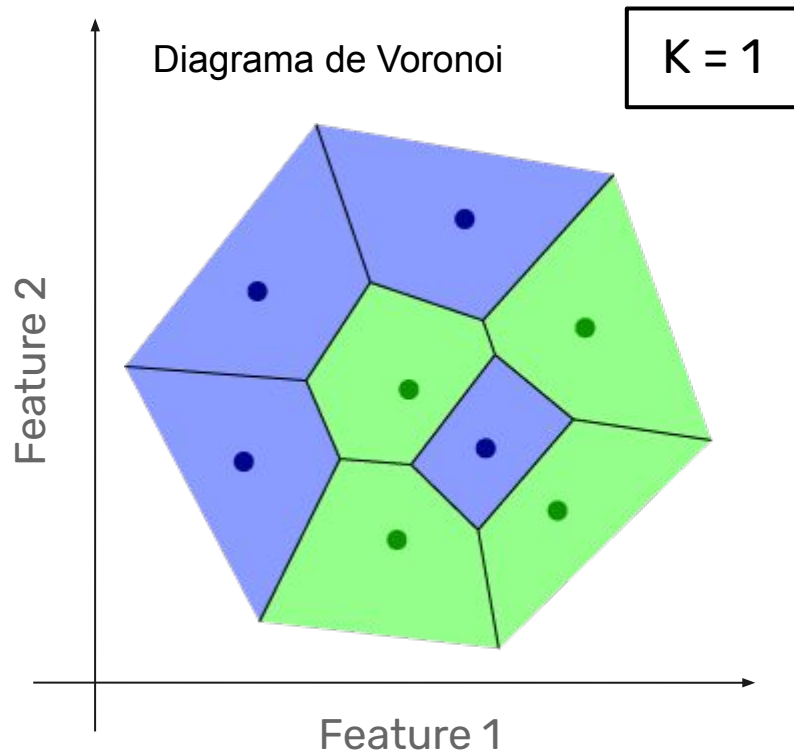
Estima la probabilidad condicional para la clase j como la fracción de puntos en N_0 cuya respuesta

$$\Pr(y=j|X=x_0)=1/K \sum_{i \in N_0} I(y_i=j)$$

Y asigna a la clase de mayor probabilidad



Fronteras de decisión según K

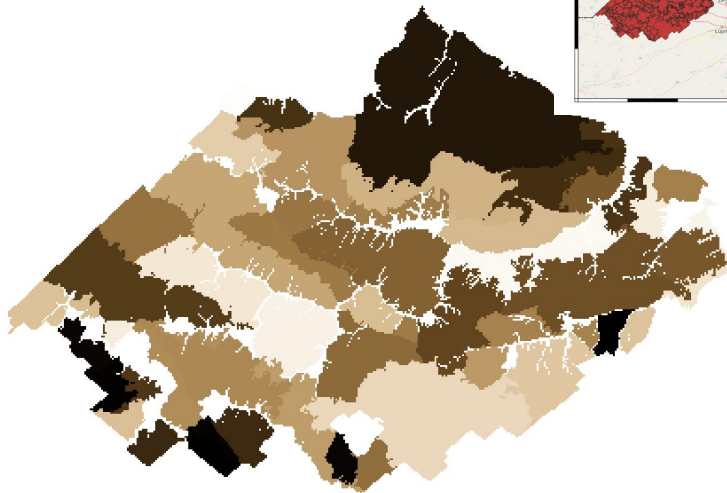


KNN Clasificación - Aplicación

Completar valores asignando la clase que domina en un entorno de la etiqueta faltante.

```
. . . . . X . . . . .  
. . X X X X X X X . .  
. X X X X X X X X .  
. X X X X X X X X .  
. X X X X X X X X .  
X X X X X 0 X X X X X  
. X X X X X X X X .  
. X X X X X X X X .  
. X X X X X X X X .  
. . X X X X X X X . .  
. . . . . X . . . . .
```

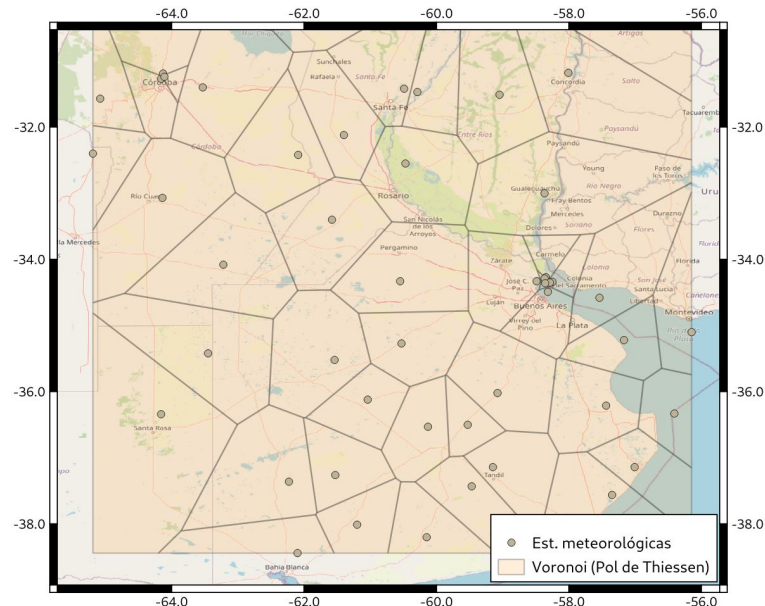
Ej.: Entorno circular de 11x11 usado para completar el dato raster que representa la clasificación de los suelos del norte bonaerense (luego de eliminar algunas áreas).



Regresión **con KNN**– Aplicación con **K=1**

Diagramas de Voronoi o polígonos de Thiessen

Asignación de los registros correspondientes a la estación meteorológica más cercana.

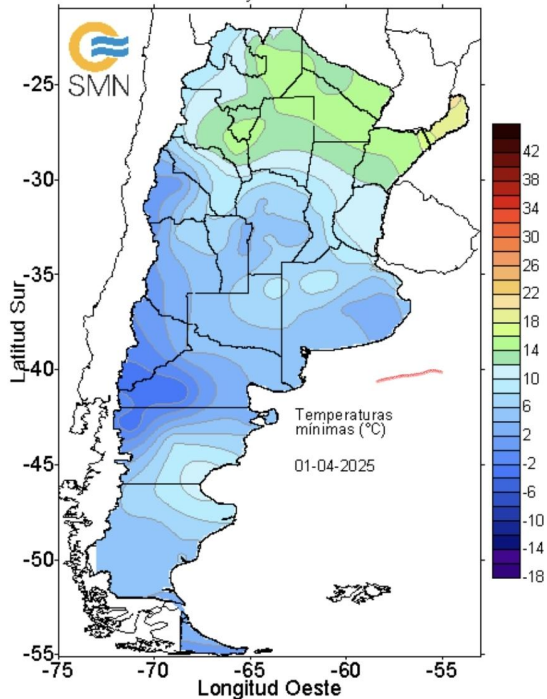


Mapas por **KNN**: Pesos inversa de la distancia

Temperatura y precipitación

Máximas Mínimas Precipitaciones

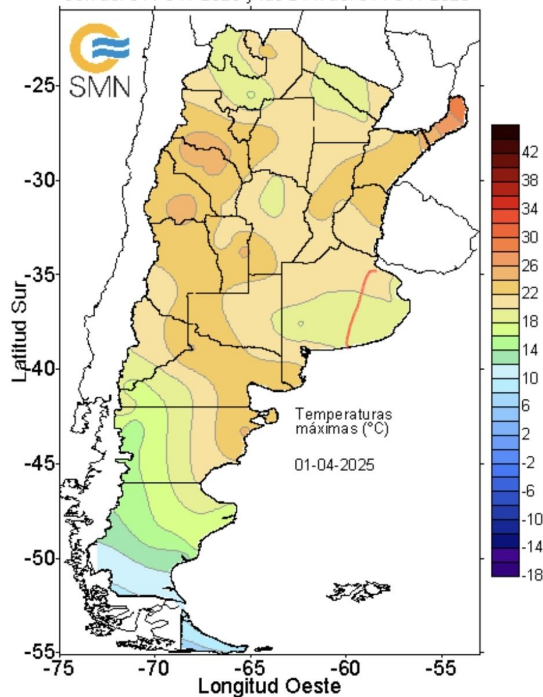
Temperaturas mínimas (°C) registradas entre las
00h del 01 / 04 / 2025 y las 24 h del 01 / 04 / 2025



Temperatura y precipitación

Máximas Mínimas Precipitaciones

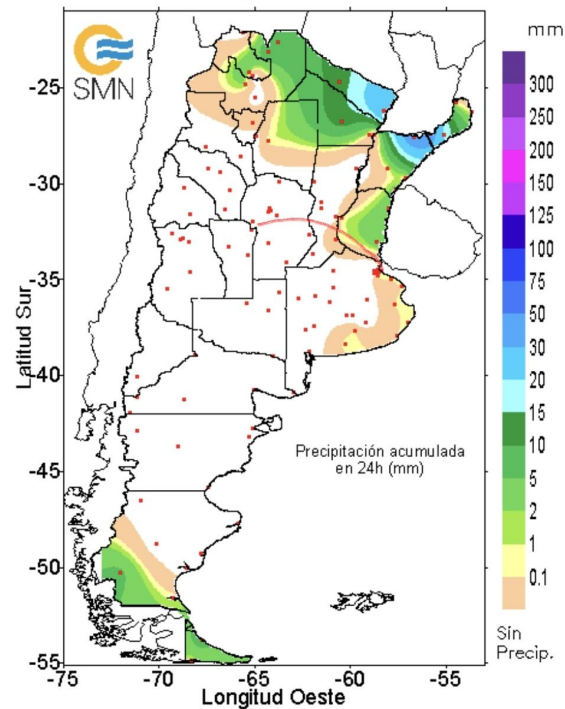
Temperaturas máximas (°C) registradas entre las
00h del 01 / 04 / 2025 y las 24 h del 01 / 04 / 2025



Temperatura y precipitación

Máximas Mínimas Precipitaciones

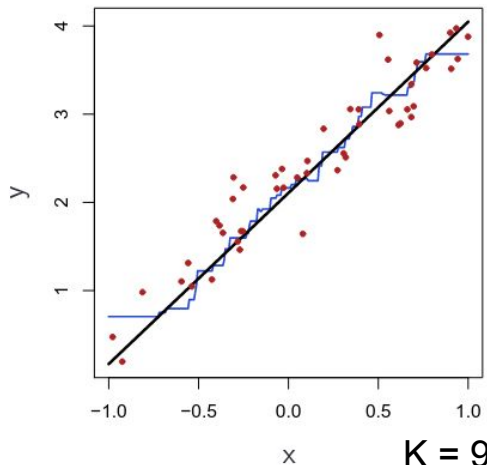
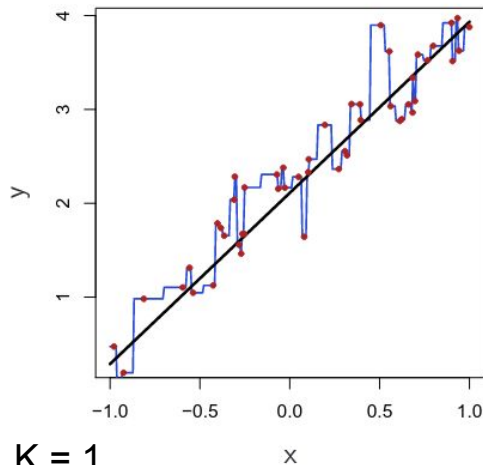
Los valores corresponden al periodo comprendido entre las
9h del 31 / 03 / 2025 y las 9h del 01 / 04 / 2025



Regresión con KNN

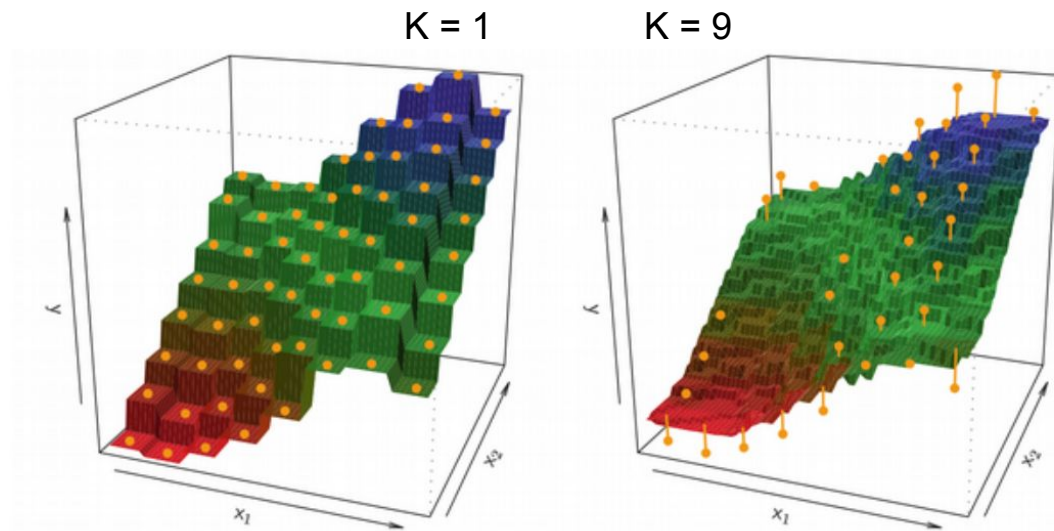
Dada una nueva instancia X_0 , identifica las K observaciones más cercanas y luego estima $f(x_0)$ como el promedio (ponderado) de las respuestas en el entorno N_0 :

$$f(x_0) = 1/K \sum_{X_i \in N_0} y_i$$



$\hat{f}(X)$ usando regresión KNN en 1D y 50 obs. La relación real se muestra en negro.

Regresión con KNN



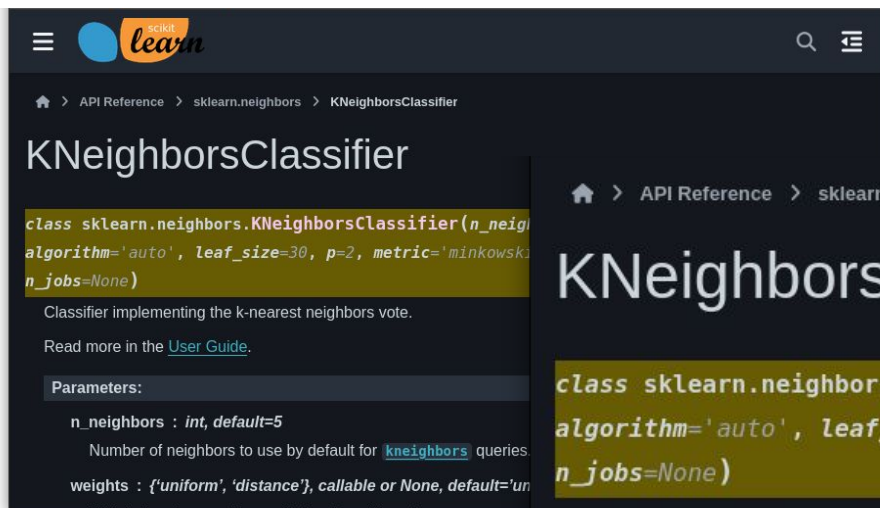
Representación de $\hat{f}(X)$ usando regresión de KNN sobre datos 2D con 64 observaciones (puntos naranjas). Izq: con $K=1$ resulta en una función escalera. Der: con $K=9$ produce un ajuste más suave.

Ventajas y Desventajas de KNN

- Simple de interpretar (discutible)
- Entrenamiento rápido.
- Lento para clasificar (predecir)
- Ocupa mucho espacio en el disco (tiene que guardar todo el set de entrenamiento)

¿Hay algún tipo de aprendizaje con KNN?

Código



API Reference > sklearn.neighbors > KNeighborsClassifier

KNeighborsClassifier

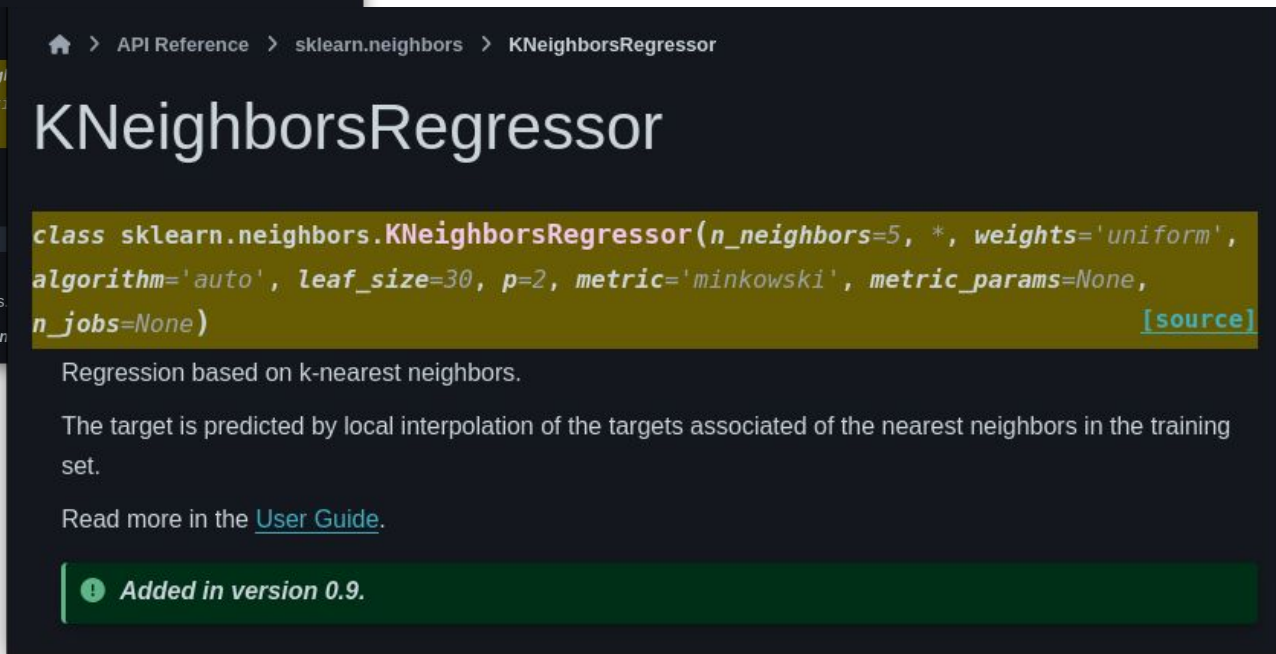
```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, *, algorithm='auto', leaf_size=30, p=2, metric='minkowski', n_jobs=None)
```

Classifier implementing the k-nearest neighbors vote.

Read more in the [User Guide](#).

Parameters:

- `n_neighbors` : int, default=5
Number of neighbors to use by default for `kneighbors` queries.
- `weights` : {'uniform', 'distance'}, callable or None, default='uniform'



API Reference > sklearn.neighbors > KNeighborsRegressor

KNeighborsRegressor

```
class sklearn.neighbors.KNeighborsRegressor(n_neighbors=5, *, weights='uniform', algorithm='auto', leaf_size=30, p=2, metric='minkowski', metric_params=None, n_jobs=None) \[source\]
```

Regression based on k-nearest neighbors.

The target is predicted by local interpolation of the targets associated of the nearest neighbors in the training set.

Read more in the [User Guide](#).

! Added in version 0.9.

Ver más en SciKit-Learn - [Nearest Neighbors](#)

Sesgo y Varianza



Aprendizaje **supervisado**

Dada una función objetivo **f** desconocida, queremos aproximarla mediante un modelo, que se suele notar **f^1** .

Entrenar un modelo

→
consiste en

ajustar sus parámetros (encontrar valores óptimos) dado un conjunto de datos.

Los algoritmos de aprendizaje automático son procedimientos para entrenar modelos a partir de un conjunto de datos

¹El sombrerito va a arriba de la **f** pero no encontramos cómo escribirlo.

Trade-off entre sesgo y varianza

No se puede construir un modelo “**perfecto**” ya que nunca pueden estar libre de errores.

$$\begin{aligned} E(Y - \hat{Y})^2 &= E[f(X) + \epsilon - \hat{f}(X)]^2 \\ &= \underbrace{[f(X) - \hat{f}(X)]^2}_{\text{Reducible}} + \underbrace{\text{Var}(\epsilon)}_{\text{Irreducible}}, \end{aligned} \quad (2.3)$$

Trade-off entre sesgo y varianza

No se puede construir un modelo “**perfecto**” ya que nunca pueden estar libre de errores.

Comprender cómo son - y cómo influyen - las diferentes fuentes de errores nos ayudará a mejorar el proceso de ajuste de datos, lo que resultará en mejores modelos y, adicionalmente, también evitará el sobreajuste (overfitting) y falta de ajuste (underfitting).

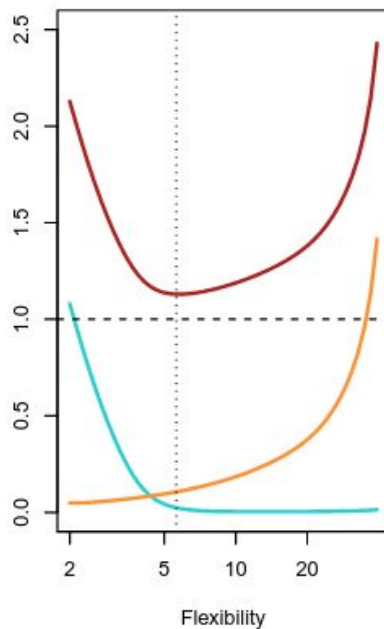
Error de predicción

El error de predicción para cualquier algoritmo de Machine Learning se puede dividir en tres partes:

Error irreducible (ruido)	Error de bias (sesgo)	Error de varianza
----------------------------------	------------------------------	--------------------------

Un poco de matemática

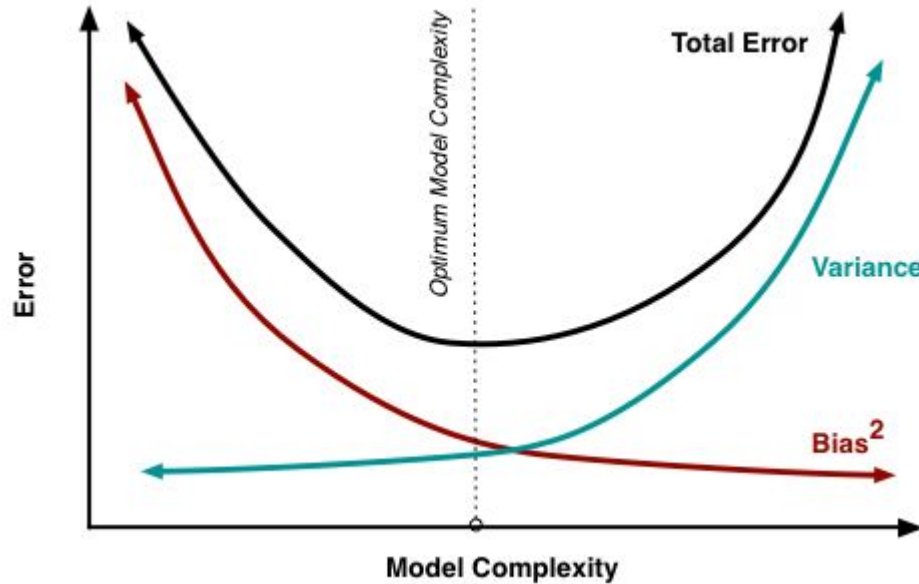
$$E \left(y_0 - \hat{f}(x_0) \right)^2 = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\epsilon)$$



Un poco de matemática

$$E \left(y_0 - \hat{f}(x_0) \right)^2 = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\epsilon)$$

- La varianza se refiere a la cantidad que **f** podría cambiar si usamos un conjunto diferente de entrenamiento.
- El bias o sesgo se refiere al error debido a la aproximación de un problema de la vida real por un modelo más simple.



Comprender el bias y la varianza es fundamental para comprender el comportamiento de los modelos de predicción, pero en general **lo que realmente importa es el error general, no la descomposición específica**. El punto ideal para cualquier modelo es el nivel de complejidad en el que el aumento en el bias es equivalente a la reducción en la varianza.

Error de predicción • Error irreducible

Error irreducible (ruido)

- No se puede reducir
- Se lo conoce como Ruido
- Proviene de factores como:
 1. Variables desconocidas que influyen en el mapeo de las variables de entrada a la variable de salida.
 1. Un conjunto de características incompletos
 2. Un problema mal enmarcado

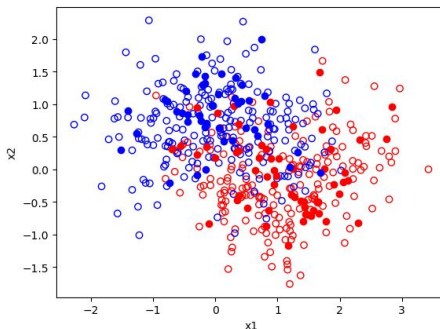
No importa cuán bien hagamos nuestro modelo, nuestros datos tendrán cierta cantidad de ruido o un error irreducible que no se puede eliminar.

Error de predicción • Error irreducible

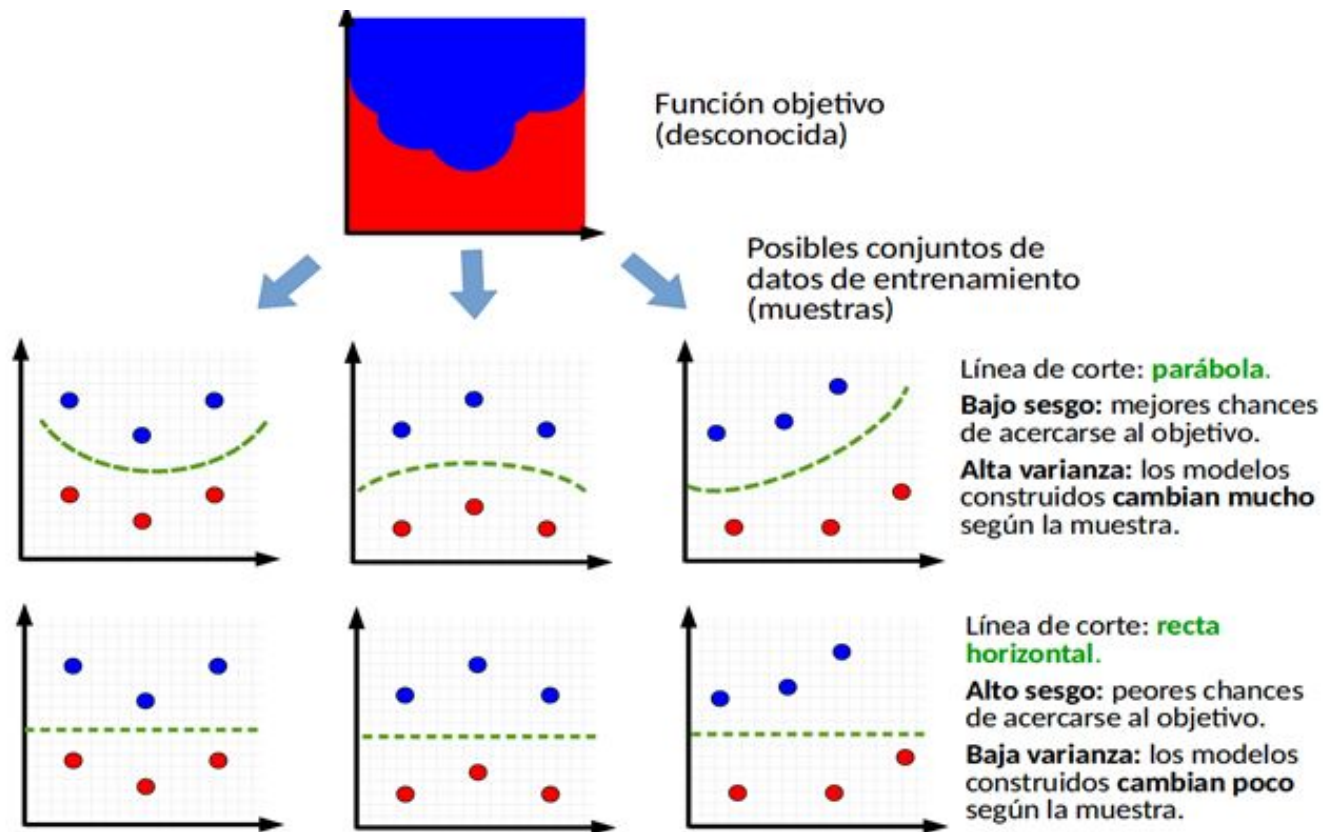
Error irreducible (ruido)

- No se puede reducir
- Se lo conoce como Ruido
- Proviene de factores como:
 1. Variables desconocidas que influyen en el mapeo de las variables de entrada a la variable de salida.
 1. Un conjunto de características incompletos
 2. Un problema mal enmarcado

No importa cuán bien hagamos nuestro modelo, nuestros datos tendrán cierta cantidad de ruido o un error irreducible que no se puede eliminar.



En cambio, los errores de sesgo y varianza se pueden reducir porque se derivan de la elección del algoritmo de entrenamiento y del modelo.



Error de predicción • Error de sesgo

Error de bias (sesgo)

- El sesgo es el error que introducimos al intentar explicar un problema del mundo real al que le correspondería un modelo complejo con un modelo bastante más simple.
- En general, modelos más flexibles y más complejos, implican menos sesgo.

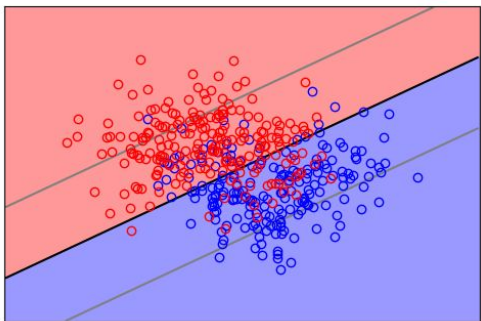
El sesgo, entonces, no es consecuencia de nuestros datos, sino del modelo elegido.

Error de predicción • Error de sesgo

Error de bias (sesgo)

- El sesgo es el error que introducimos al intentar explicar un problema del mundo real al que le correspondería un modelo complejo con un modelo bastante más simple.
- En general, modelos más flexibles y más complejos, implican menos sesgo.

El sesgo, entonces, no es consecuencia de nuestros datos, sino del modelo elegido.

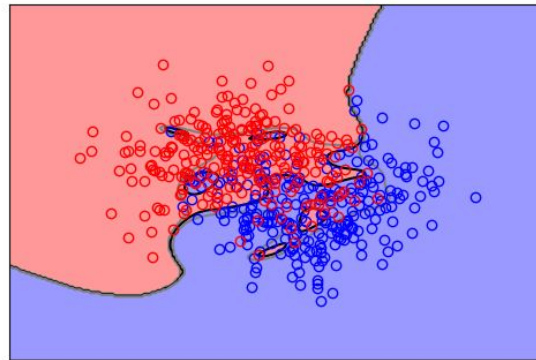
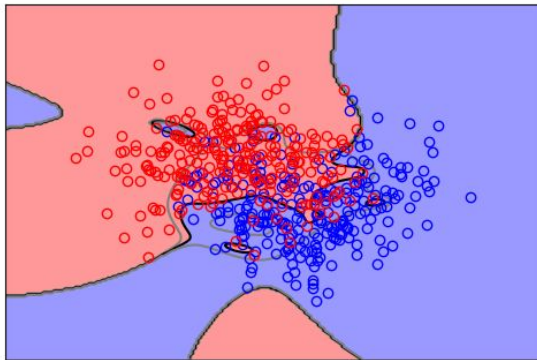


Error de predicción • Error de varianza

Error de varianza

- La varianza es la cantidad en la que cambiaría mi predicción si hubiéramos entrenado nuestro modelo con un conjunto de datos diferente.
- Por ejemplo, un modelo que se ajuste mucho a unos datos sufrirá una varianza considerable al cambiar dichos datos, y viceversa.

En general, modelos más flexibles y complejos implican una mayor varianza.



Entonces...

Cuanto más flexible sea nuestro modelo, la varianza aumentará, y el sesgo disminuirá.

Por lo tanto, debemos prestar atención a estas características, ya que pueden introducir mucho ruido en nuestro modelo y, por lo tanto, hacerlo menos exacto.

Con un análisis de ambos podemos evaluar el rendimiento de nuestro modelo.

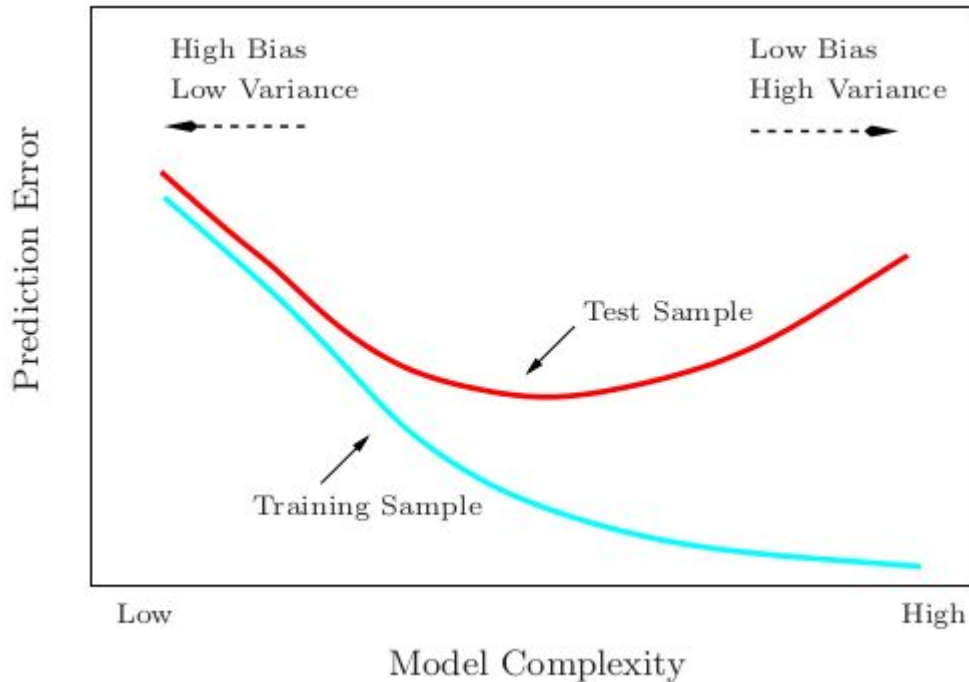
Parámetros

Hacen referencia a las variables que se ajustan automáticamente durante el proceso de entrenamiento. Esta es la parte en el que el modelo “aprende solo”.

Más en detalle: son elegidos por el algoritmo de optimización para minimizar la función de pérdida medida sobre el set de datos de entrenamiento.

Hiperparámetros

Son elegidos antes de entrenar el modelo. El criterio de elección es para conseguir un modelo que *generalice mejor*.



[Fuente](#)

En el gráfico, si nos movemos de izquierda a derecha:

- Aumenta la complejidad de nuestro modelo
- Baja el sesgo y aumenta la varianza.
- Hasta que llega un momento en el que el error en los datos de test empieza a aumentar mientras que el de entrenamiento sigue disminuyendo. Ese punto mínimo de error en los datos de test nos indica el nivel de complejidad óptimo para nuestro modelo.

Resumen

Modelo sesgado: No logra capturar la forma de los datos. En general, tiene desempeño muy similar en el set de entrenamiento y de validación. Asociado al underfitting.

Modelo con mucha varianza: Demasiado ajustado a los datos. Tiene desempeño muy bueno en el set de entrenamiento y malo en el de validación. Asociado al overfitting.

¿Cómo diagnosticamos sesgo y varianza?

Curva de validación/complejidad: Score en función de la complejidad. Sirve para ver regiones de baja complejidad (sesgo, underfitting) y demasiada complejidad (alta varianza, overfitting)

Curva de aprendizaje: Score en función de la cantidad de datos. Sirve para ver, dado un modelo fijo, cómo reacciona a distintos tamaño del dataset. En particular, útil para diagnosticar alta varianza o modelo muy complejo (dado el tamaño de nuestro dataset).

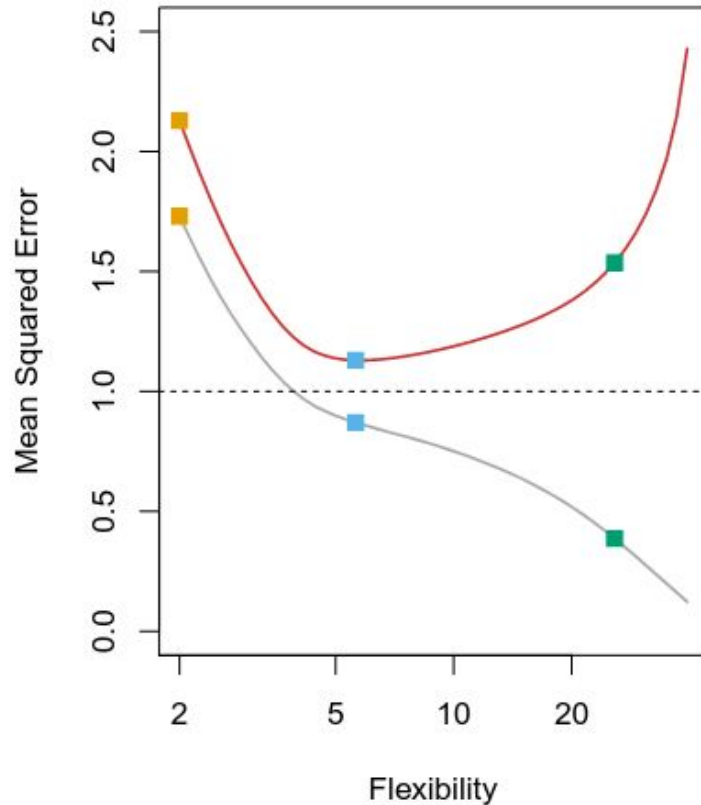
Navaja de Ockham

En igualdad de condiciones (por ejemplo, igual desempeño), elegir la explicación (modelo) más simple.

¿Por qué? Simplemente porque esperamos que generalice mejor.

Es un principio metodológico.

Curva de Complejidad



Mayor flexibilidad → Mayor complejidad

- Mayor profundidad en el árbol de decisión
- Atención: Menos vecinos (K chico) en K-NN
- Mayor grado polinómico en los atributos (más adelante...)

Ejemplo en KNN: Hiperparámetros

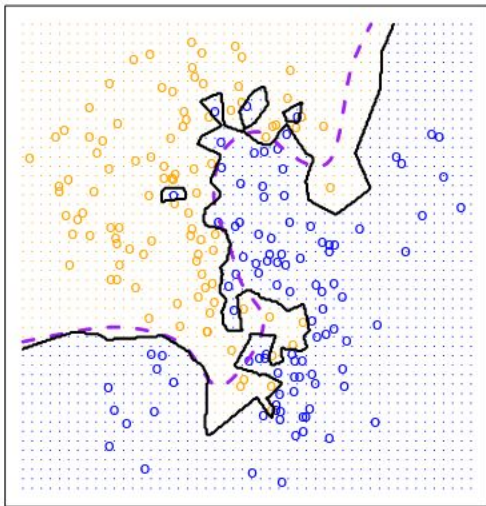
- K
- Métrica de distancia
- Pesos

¿Cuál controla el sub y sobreajuste?

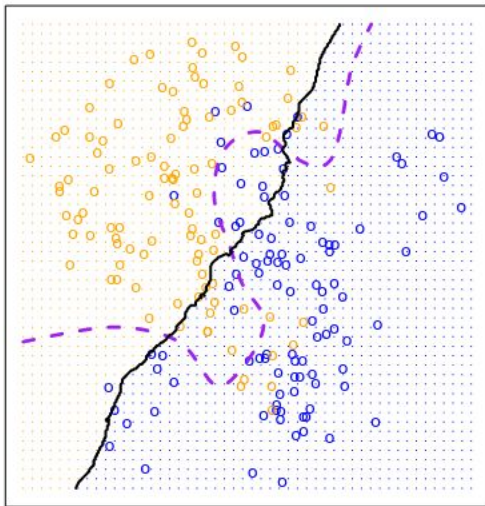
Ejemplo en KNN: Clasificación

[Fuente: An introduction to Statistical Learning](#)

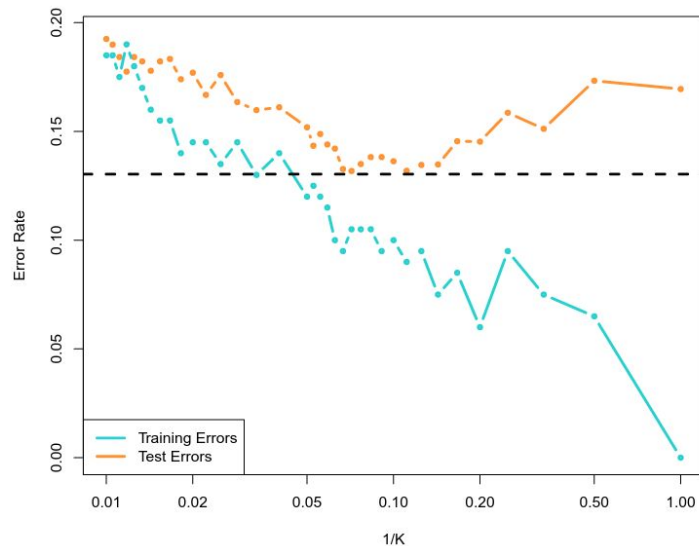
KNN: $K=1$



KNN: $K=100$



Con $K = 1$, la frontera de decisiones es muy flexible, mientras que con $K=100$ no lo es suficientemente... La frontera de Bayes (óptima) se muestra en violeta.



Errores de prueba y entrenamiento en función de la flexibilidad ($1/K$). El trazo en negro representa la tasa de error de Bayes

Ejemplo en KNN: Regresión

Arr-izq: Configuración levemente no lineal, ajuste de KNN con $K = 1$ (azul) y $K = 9$ (rojo) Arr-Der: MSE de prueba (regresión lineal señalada con puntos).

Abajo: Como en el panel superior pero para una relación fuertemente no-lineal.

[Fuente: An introduction to Statistical Learning](#)

